Moveit Positive Kinematics Design

Moveit Positive Kinematics Design
- 1. Usage Environment
- 2. Driving the Real Machine
  - 3. Start MoveIt2
    - 3.1 Forward Kinematics Design
      - Launch Command
    - 3.2. Source Code Analysis

1. Usage Environment

ROS2: Humble

Enter the commands in the tutorial into the accompanying virtual machine to run the function.

2. Driving the Real Machine

Driving the real machine is achieved by subscribing to the Moveit2 `/joint_states` topic, converting the joint state information of the robotic arm into control data for the actual robotic arm.

Note: Since the real robotic arm does not have obstacle avoidance capabilities, some poses may encounter obstacles; therefore, the planned robotic arm movements should be as reasonable as possible and avoid locations with obstacles.

(It is recommended to use preset poses for demonstrating the driving of the real machine.)

The motherboard and virtual machine need to be configured for multi-machine communication. The robotic arm and virtual machine need to be on the same network, and the virtual machine's `ros_domain_id` must be consistent with the `ros_domain_id` in the Docker container. (Both are pre-configured with the same ros_domain_id at the factory; only the virtual machine and the robotic arm need to be on the same network.)

```
# Start this command in Docker,
ros2 run dofbot_ctrl SimulateToArm
```

3. Start MoveIt2

```
# Start this command in the virtual machine,
ros2 launch dofbot_moveit demo.launch.py
```

```
yahboom@yahboom-VM:~$ ros2 launch dofbot_moveit demo.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-03-13-14-31-56-635718-yahboom-VM-9623
[INFO] [launch]: Default logging verbosity is set to INFO
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
[INFO] [robot_state_publisher-1]: process started with pid [9625]
[INFO] [move_group-2]: process started with pid [9627]
[INFO] [rviz2-3]: process started with pid [9629]
[INFO] [ros2_control_node-4]: process started with pid [9631]
[INFO] [spawner-5]: process started with pid [9633]
[INFO] [spawner-6]: process started with pid [9635]
[INFO] [spawner-7]: process started with pid [9637]
[robot_state_publisher-1] [WARN] [1773383518.815694424] [kdl_parser]: The root link base_link has an inertia specified in the URDF,
but KDL does not support a root link with an inertia.  As a workaround, you can add an extra dummy link to your URDF.
[robot_state_publisher-1] [INFO] [1773383518.815824354] [robot_state_publisher]: got segment Camera_Link
[robot_state_publisher-1] [INFO] [1773383518.815886691] [robot_state_publisher]: got segment Gripping_point_Link
[robot_state_publisher-1] [INFO] [1773383518.815895376] [robot_state_publisher]: got segment Llink1_Link
[robot_state_publisher-1] [INFO] [1773383518.815901628] [robot_state_publisher]: got segment Llink2_Link
[robot_state_publisher-1] [INFO] [1773383518.815907310] [robot_state_publisher]: got segment Llink3_Link
```

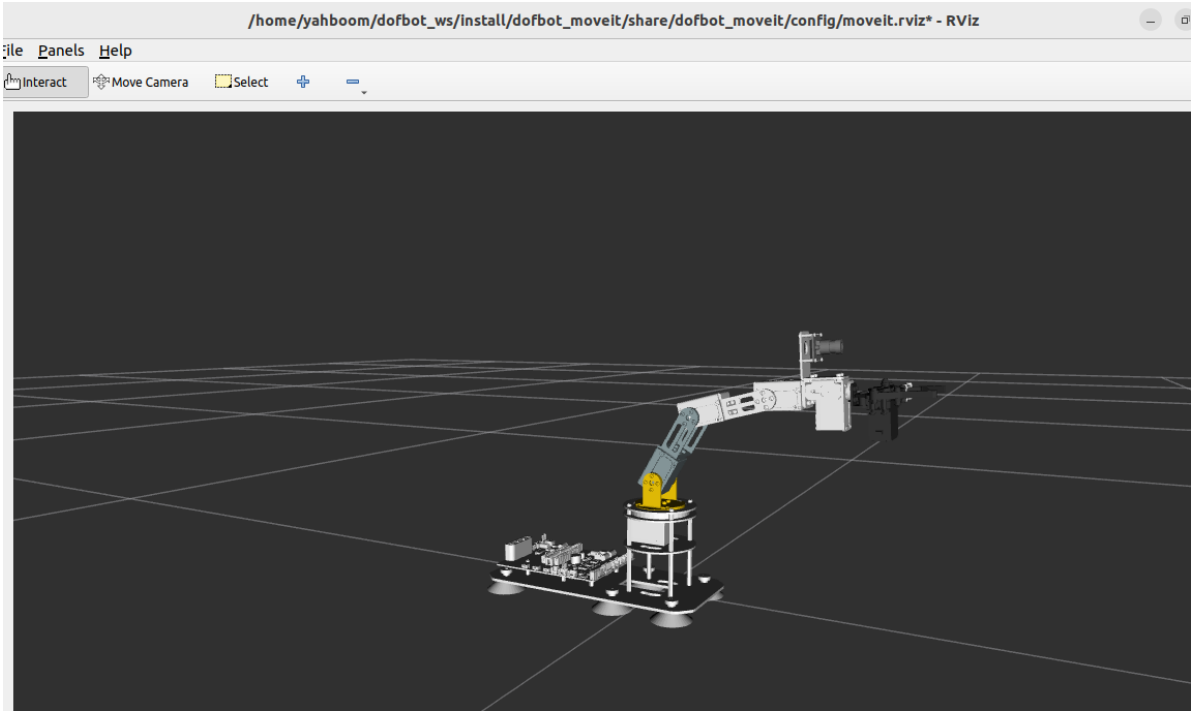
3.1 Forward Kinematics Design

Forward kinematics refers to giving the robotic arm a target pose (angles of each joint), and then MoveIt automatically plans to reach the target pose.

Launch Command

Once the robotic arm has successfully loaded in MoveIt and the message `You can start planning now!` appears, execute the following command: the robotic arm will automatically plan a path to the target pose.

```
# Execute this command within the virtual machine
ros2 run dofbot_moveit set_target_joints
```



3.2. Source Code Analysis

Source Code Path:

```
#Path in the Virtual Machine
~/dofbot_ws/src/dofbot_moveit/src/set_target_joints.cpp
```

```
#include <rclcpp/rclcpp.hpp>
#include <moveit/move_group_interface/move_group_interface.h>
#include <moveit/planning_scene_interface/planning_scene_interface.h>
#include <geometry_msgs/msg/pose.hpp>
#include <random>
#include <moveit_visual_tools/moveit_visual_tools.h>
#include <rclcpp/rclcpp.hpp>
#include <moveit_msgs/msg/display_trajectory.hpp>
#include <vector>
#include <moveit/robot_trajectory/robot_trajectory.h>
#include <moveit/robot_state/robot_state.h>
#include <moveit/robot_model/robot_model.h>
#include <moveit/robot_model_loader/robot_model_loader.h>


class RandomMoveIt2Control : public rclcpp::Node
{
public:
  RandomMoveIt2Control()
    : Node("random_moveit2_control")
  {
    // Initialize other content
    RCLCPP_INFO(this->get_logger(), "Initializing RandomMoveIt2Control.");
  }

  void initialize()
  {
    int max_attempts = 5;  // Maximum number of planning attempts
    int attempt_count = 0;  // Current attempt count

      // Load the robot model using RobotModelLoader
    robot_model_loader::RobotModelLoader
robot_model_loader(shared_from_this(),"robot_description");
    const moveit::core::RobotModelPtr& robot_model =
robot_model_loader.getModel();
    // Initialize move_group_interface_ and planning_scene_interface_ in this
function.
    move_group_interface_ =
std::make_shared<moveit::planning_interface::MoveGroupInterface>
(shared_from_this(), "arm_group");
    planning_scene_interface_ =
std::make_shared<moveit::planning_interface::PlanningSceneInterface>();

    const moveit::core::JointModelGroup* joint_model_group = robot_model-
>getJointModelGroup("arm_group");

    moveit_visual_tools::MoveItVisualTools visual_tools_(shared_from_this(),
"base_link", "arm_group",move_group_interface_->getRobotModel());

    move_group_interface_->setNumPlanningAttempts(10);  // Set the maximum
number of planning attempts to 10
    move_group_interface_->setPlanningTime(5.0);         // Set the maximum
planning time per attempt to 5 seconds.
    move_group_interface_->allowReplanning(true);
    move_group_interface_->setReplanAttempts(5);
    // Set up the random number generator
```

```
    while (attempt_count < max_attempts)
    {
        attempt_count++;
        //std::vector<double> target_joints = {0.0, 0.5, -1.5, 0.5, 0}; //
Target joint angles (units: radians)
        std::vector<double> target_joints = {0.0, 0.52, 0.786, 0.20, 0}; //
Target joint angles (units: radians)
        // Set target joint space values
        move_group_interface_->setJointValueTarget(target_joints);
            // Plan Path
        moveit::planning_interface::MoveGroupInterface::Plan my_plan;
        bool success = (move_group_interface_->plan(my_plan) ==
moveit::core::MoveItErrorCode::SUCCESS);
        if (success)
        {
          // Visualize the trajectory in RViz

visual_tools_.publishTrajectoryLine(my_plan.trajectory_,move_group_interface_-
>getRobotModel()-
>getLinkModel("Gripping_point_Link"),joint_model_group,rviz_visual_tools::LIME_G
REEN);
        visual_tools_.trigger();
        RCLCPP_INFO(this->get_logger(), "Planning succeeded, moving the arm.");
        move_group_interface_->execute(my_plan);
        return;
        }
        else
        {
        RCLCPP_ERROR(this->get_logger(), "Planning failed!");
        }
    }

    RCLCPP_ERROR(this->get_logger(), "Exit!");


  }

private:
  std::shared_ptr<moveit::planning_interface::MoveGroupInterface>
move_group_interface_;
  std::shared_ptr<moveit::planning_interface::PlanningSceneInterface>
planning_scene_interface_;
  std::shared_ptr<moveit_visual_tools::MoveItVisualTools> visual_tools_;
};

int main(int argc, char **argv)
{
  rclcpp::init(argc, argv);
  auto node = std::make_shared<RandomMoveIt2Control>();

  // Lazy Initialization
  node->initialize();

  rclcpp::spin(node);
  rclcpp::shutdown();
  return 0;
}
```